Lecture 40 Practice Solutions: Deep Learning (Part-5)

Complete Mathematical Derivations & Code Solutions

Solution 1: CIFAR-10 Normalization Mechanics

1. **Mathematical Derivation:** `transforms.ToTensor()` maps integer pixels $p \in [0, 255]$ to floating-point values:

$$x = \frac{p}{255.0} \in [0.0, 1.0]$$

The `transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))` operation applies the channel-wise affine transformation:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} = \frac{x - 0.5}{0.5} = 2x - 1$$

   - At lower bound ($x = 0.0$): $x_{\text{norm}} = 2(0.0) - 1 = -1.0$
   - At midpoint ($x = 0.5$): $x_{\text{norm}} = 2(0.5) - 1 = 0.0$
   - At upper bound ($x = 1.0$): $x_{\text{norm}} = 2(1.0) - 1 = +1.0$

   Therefore, the transformed pixel values lie strictly in the range $[-1.0, 1.0]$.

2. **Why Zero-Centered Data is Critical:** When inputs $x_i \geq 0$ are strictly positive (e.g., $[0, 1]$), the gradient of the loss with respect to weights w_i in layer 1 is:

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial z} x_i$$

   Because $x_i > 0$ for all inputs, all weight gradients share the sign of $\frac{\partial \mathcal{L}}{\partial z}$. Consequently, during backpropagation, all weights connected to a neuron must either increase together or decrease together, forcing gradient descent into an inefficient "zig-zag" trajectory. Zero-centering eliminates this systemic gradient bias.

Solution 2: Feature Map Spatial Shape Calculation

Formula for spatial output dimension after 2D convolution:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 2P}{S} \right\rfloor + 1$$

For $K = 3, P = 1, S = 1$:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - 3 + 2(1)}{1} \right\rfloor + 1 = H_{\text{in}}$$

Thus, same-padding preserves spatial dimensions. MaxPool with $K = 2, S = 2$ strictly halves both spatial dimensions:

$$H_{\text{pool}} = \left\lfloor \frac{H_{\text{out}}}{2} \right\rfloor$$

- **Input:** `(Batch, 3, 32, 32)`
- **Block 1:**
  - `Conv2d(3, 32, K=3, P=1)` $\rightarrow$ `(Batch, 32, 32, 32)`
  - `ReLU()` $\rightarrow$ `(Batch, 32, 32, 32)`
  - `MaxPool2d(2, 2)` $\rightarrow$ `(Batch, 32, 16, 16)`
- **Block 2:**
  - `Conv2d(32, 64, K=3, P=1)` $\rightarrow$ `(Batch, 64, 16, 16)`
  - `ReLU()` $\rightarrow$ `(Batch, 64, 16, 16)`
  - `MaxPool2d(2, 2)` $\rightarrow$ `(Batch, 64, 8, 8)`
- **Block 3:**
  - `Conv2d(64, 128, K=3, P=1)` $\rightarrow$ `(Batch, 128, 8, 8)`
  - `ReLU()` $\rightarrow$ `(Batch, 128, 8, 8)`
  - `MaxPool2d(2, 2)` $\rightarrow$ `(Batch, 128, 4, 4)`
- **Flattening:**

$$\text{Nodes} = C_{\text{out}} \times H_{\text{final}} \times W_{\text{final}} = 128 \times 4 \times 4 = \mathbf{2,048} \text{ features}$$

Solution 3: Parameter Count Analysis

The parameter formula for `Conv2d(C_in, C_out, K)` with bias:

$$\text{Params}_{\text{conv}} = (K \times K \times C_{\text{in}} + 1) \times C_{\text{out}}$$

The parameter formula for `Linear(in_dim, out_dim)` with bias:

$$\text{Params}_{\text{linear}} = (\text{in_dim} + 1) \times \text{out_dim}$$

1. **Conv1 (3 to 32, 3x3):**

$$(3 \times 3 \times 3 + 1) \times 32 = (27 + 1) \times 32 = 28 \times 32 = \mathbf{896}$$

2. **Conv2 (32 to 64, 3x3):**

$$(3 \times 3 \times 32 + 1) \times 64 = (288 + 1) \times 64 = 289 \times 64 = \mathbf{18,496}$$

3. **Conv3 (64 to 128, 3x3):**

$$(3 \times 3 \times 64 + 1) \times 128 = (576 + 1) \times 128 = 577 \times 128 = \mathbf{73,856}$$

4. **Linear1 (2048 to 256):**

$$(2048 + 1) \times 256 = 2049 \times 256 = \mathbf{524,544}$$

5. **Linear2 (256 to 10):**

$$(256 + 1) \times 10 = 257 \times 10 = \mathbf{2,570}$$

6. **Total Model Parameters:**

$$896 + 18,496 + 73,856 + 524,544 + 2,570 = \mathbf{620,362} \text{ parameter}$$

Solution 4: Model Evaluation Mode & No-Grad Context

1. `model.eval()` **Effects:**
   - **Dropout:** During training, dropout randomly zeroes out activations with probability p and scales remaining activations by $\frac{1}{1-p}$. In `eval()` mode, dropout is completely deactivated; all activations pass through multiplied by 1.
   - **Batch Normalization:** During training, BatchNorm computes batch mean and variance and updates running tracking statistics. In `eval()` mode,

BatchNorm stops updating running statistics and uses frozen population mean and variance.

2. `with torch.no_grad()` **Purpose:**
    - Disables Autograd engine tracking. Intermediate feature maps are not recorded into the dynamic computation graph.
    - **Consequences if omitted:** Intermediate activations remain in GPU/RAM memory in anticipation of `backward()`, causing large unnecessary memory allocations (often triggering Out-Of-Memory errors on large test sets) and slowing down inference by $30 - 50\%$.

Solution 5: VGG Factorization and Receptive Field Equivalence

1. **Receptive Field Proof:** The effective receptive field RF_l of layer l with filter size k and stride $s = 1$ is given by:

$$RF_l = RF_{l-1} + (k - 1)$$

    - Layer 1 (3×3): $RF_1 = 1 + (3 - 1) = 3$
    - Layer 2 (3×3): $RF_2 = 3 + (3 - 1) = 5$ Thus, two stacked 3×3 layers cover an effective 5×5 spatial area on the input.

2. **Parameter Ratio:**
    - Single 5×5 layer: $5 \times 5 \times C^2 = 25C^2$ weights.
    - Two stacked 3×3 layers: $2 \times (3 \times 3 \times C^2) = 18C^2$ weights.

$$\text{Ratio} = \frac{18C^2}{25C^2} = \frac{18}{25} = 0.72 \implies \mathbf{28\%} \text{ parameter reducti}$$

    In addition, stacking two layers introduces two non-linear activations (ReLU) instead of one, significantly boosting model representational capacity.

Solution 6: ResNet Residual Learning & Gradient Highway

1. **Formulation:**

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

2. **Derivative:** Using the chain rule on loss $\mathcal{E}$:

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}} = \frac{\partial \mathcal{E}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{E}}{\partial \mathbf{y}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right) = \frac{\partial \mathcal{E}}{\partial \mathbf{y}} \frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \frac{\partial \mathcal{E}}{\partial \mathbf{y}}$$

3. **Elimination of Vanishing Gradients:** Notice the second term $\frac{\partial \mathcal{E}}{\partial \mathbf{y}}$! Even if the learned residual transformations $\frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ vanish towards zero, the gradient flows directly backward through the identity connection $\mathbf{I}$ completely unattenuated:

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}} \approx \frac{\partial \mathcal{E}}{\partial \mathbf{y}} \neq \mathbf{0}$$

This creates an uninterrupted gradient highway from output to input.

Solution 7: Stemming vs Lemmatization

1. **Word Transformations:**
   - `"running"`: Stemming $\rightarrow$ `"run"` | Lemmatization (Verb) $\rightarrow$ `"run"`
   - `"better"`: Stemming $\rightarrow$ `"better"` | Lemmatization (Adj) $\rightarrow$ `"good"`
   - `"flies"`: Stemming $\rightarrow$ `"fli"` (rule-based suffix chop) | Lemmatization (Noun) $\rightarrow$ `"fly"`
   - `"corpora"`: Stemming $\rightarrow$ `"corpora"` | Lemmatization (Noun) $\rightarrow$ `"corpus"`
2. **Stemming Failure Modes:**
   - **Over-stemming:** Words of distinct semantic origins chopped to the same base stem (e.g., `"universe"`, `"universal"`, `"university"` all chopped to `"univers"`).
   - **Under-stemming:** Words sharing the same semantic root chopped to different stems (e.g., `"alumnus"`, `"alumni"`, `"alumnae"` not collapsed).

Solution 8: One-Hot Sparsity & Dot-Product Blindness

1. **Memory Calculation:** For a 500-word document with $|\mathcal{V}| = 100,000$:
   - Total elements = $500 \times 100,000 = 50,000,000$ floats.
   - Each float32 is 4 bytes:

$$\text{Bytes} = 50,000,000 \times 4 = 200,000,000 \text{ bytes} \approx \mathbf{190.73}\ \text{M}$$

For a single short document, this is an immense waste of RAM.

2. **Orthogonality Proof:** Let $\mathbf{e}_k$ and $\mathbf{e}_q$ be the one-hot vectors for `"king"` and `"queen"`. Since $k \neq q$:

$$\mathbf{e}_k \cdot \mathbf{e}_q = \sum_{j=1}^{|\mathcal{V}|} \mathbf{e}_k[j]\mathbf{e}_q[j] = 0$$

$$\text{Cosine Similarity} = \frac{\mathbf{e}_k \cdot \mathbf{e}_q}{\|\mathbf{e}_k\|\|\mathbf{e}_q\|} = \frac{0}{1 \times 1} = 0$$

One-hot encoding treats all distinct words as completely orthogonal and equidistant, capturing zero semantic relationship.

Solution 9: Bag-of-Words Document-Term Matrix

1. **Corpus:**
   - D_1: `"deep learning models learn patterns"`
   - D_2: `"deep neural models learn deep patterns"`

   Sorted unique vocabulary $\mathcal{V}$ (size 6): `['deep', 'learn', 'learning', 'models', 'neural', 'patterns']`

2. **Document-Term Matrix (2×6):**
   - D_1: deep: 1, learn: 1, learning: 1, models: 1, neural: 0, patterns: 1
   - D_2: deep: 2, learn: 1, learning: 0, models: 1, neural: 1, patterns: 1

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 1 \\ 2 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

3. **Handling Out-Of-Vocabulary (D_3):** `"transformers generate text"` contains zero vocabulary words. In standard BoW, all counts are 0:

$$\mathbf{v}_{D_3} = [0, 0, 0, 0, 0, 0]$$

The document is completely unrepresented (all-zero vector).

Solution 10: N-Gram Negation Handling

1. **Why BoW Fails:**
   - S_1: `['bad', 'film', 'good', 'not', 'was']`

- S_2: `['bad', 'film', 'good', 'not', 'was']` Both documents contain the exact same words with identical counts:

$$\mathbf{v}_{S_1} = \mathbf{v}_{S_2} = [1, 1, 1, 1, 2]$$

A linear classifier produces identical predictions despite opposite meanings!

2. **Bigrams ($n = 2$):**

- S_1 bigrams: `"The film"`, `"film was"`, `"was not"`, **`"not good"`**, `"good it"`, `"it was"`, **`"was bad"`**.
- S_2 bigrams: `"The film"`, `"film was"`, `"was not"`, **`"not bad"`**, `"bad it"`, `"it was"`, **`"was good"`**.

The bigrams `"not good"` vs `"not bad"` explicitly isolate the negation phrases, allowing the classifier to assign negative weights to `"not good"` and positive weights to `"not bad"`.

Solution 11: Step-by-Step TF-IDF Calculation

Corpus:

- D_1: `"cat sat mat"` (Length = 3)
- D_2: `"cat cat cat"` (Length = 3)
- D_3: `"dog bark loud"` (Length = 3)
- D_4: `"cat dog friends"` (Length = 3) Total documents $N = 4$.

1. **TF Calculation:**

- $\text{TF}(\text{"cat"}, D_2) = \frac{3}{3} = \mathbf{1.0}$
- $\text{TF}(\text{"cat"}, D_1) = \frac{1}{3} \approx \mathbf{0.3333}$

2. **IDF Calculation:**

- `"cat"` appears in $D_1, D_2, D_4 \implies \text{DF}(\text{"cat"}) = 3$:

$$\text{IDF}(\text{"cat"}) = \log_{10}\left(\frac{4}{3}\right) = \log_{10}(1.3333) \approx \mathbf{0.1249}$$

- `"mat"` appears in D_1 only $\implies \text{DF}(\text{"mat"}) = 1$:

$$\text{IDF}(\text{"mat"}) = \log_{10}\left(\frac{4}{1}\right) = \log_{10}(4) \approx \mathbf{0.6021}$$

3. **TF-IDF Weights:**
   - $\text{TF-IDF}(\text{"cat"}, D_2) = 1.0 \times 0.1249 = \mathbf{0.1249}$
   - $\text{TF-IDF}(\text{"mat"}, D_1) = \frac{1}{3} \times 0.6021 \approx \mathbf{0.2007}$ Notice that even though `"cat"` appears 3 times in D_2, `"mat"` in D_1 achieves a higher TF-IDF score because `"mat"` is far more distinctive and informative across the corpus!

Solution 12: Word2Vec: CBOW vs Skip-Gram Dynamics

1. **Architectural Difference:**
   - **CBOW:** Predicts center word w_t given context words within window c: $\max \sum_t \log P(w_t \mid w_{t-c}, \ldots, w_{t+c})$.
   - **Skip-Gram:** Predicts context words given center word w_t: $\max \sum_t \sum_{j \neq 0} \log P(w_{t+j} \mid w_t)$.
2. **Cosine Similarity Computation:** Given $\mathbf{u} = [0.8, 0.6, 0.0]$ and $\mathbf{v} = [0.6, 0.8, 0.0]$:
   - Dot product: $\mathbf{u} \cdot \mathbf{v} = (0.8)(0.6) + (0.6)(0.8) + (0)(0) = 0.48 + 0.48 = 0.96$
   - Norm $\|\mathbf{u}\|_2 = \sqrt{0.8^2 + 0.6^2 + 0^2} = \sqrt{0.64 + 0.36} = 1.0$
   - Norm $\|\mathbf{v}\|_2 = \sqrt{0.6^2 + 0.8^2 + 0^2} = \sqrt{0.36 + 0.64} = 1.0$

$$\text{Cosine Similarity} = \frac{0.96}{1.0 \times 1.0} = \mathbf{0.96}$$

3. **Vector Analogy:** Word2Vec embeddings map linguistic relationships to parallel direction vectors in continuous space. The gender displacement vector $\vec{v}_{\text{Woman}} - \vec{v}_{\text{Man}}$ represents the transformation from male to female gender. Adding this offset to $\vec{v}_{\text{King}}$ shifts the vector along the gender axis to land closest to $\vec{v}_{\text{Queen}}$.

Solution 13: RNN Parameter Count & Time Unrolling

1. **Parameter Shapes and Counts:**
   - $\mathbf{W}_{xh} \in \mathbb{R}^{256 \times 100} \implies 256 \times 100 = 25,600$
   - $\mathbf{W}_{hh} \in \mathbb{R}^{256 \times 256} \implies 256 \times 256 = 65,536$
   - $\mathbf{b}_h \in \mathbb{R}^{256} \implies 256$
   - $\mathbf{W}_{hy} \in \mathbb{R}^{10 \times 256} \implies 10 \times 256 = 2,560$
   - $\mathbf{b}_y \in \mathbb{R}^{10} \implies 10$

2. **Total Model Parameters:**

$$25,600 + 65,536 + 256 + 2,560 + 10 = \mathbf{93,962} \text{ parameters}$$

3. **Sequence Length Invariance:** The parameter count is **completely invariant** to sequence length T. Due to **parameter sharing**, the exact same matrices ($\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy}$) are applied recursively at every single time step $t \in \{1, \dots, T\}$.

Solution 14: BPTT Gradient Vanishing Mechanics & Gradient Clipping

1. **Matrix Powers in BPTT:** Recall: $\mathbf{h}_j = \tanh(\mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{W}_{xh}\mathbf{x}_j + \mathbf{b}_h)$. The Jacobian is:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \text{diag}(1 - \mathbf{h}_j^2)\mathbf{W}_{hh}^T$$

Multiplying across $T - k$ steps yields:

$$\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \prod_{j=k+1}^{t} \text{diag}(1 - \mathbf{h}_j^2)\mathbf{W}_{hh}^T$$

This expression repeatedly multiplies the matrix $\mathbf{W}_{hh}^T$ by itself $t - k$ times (analogous to $(\mathbf{W}_{hh}^T)^{t-k}$).

2. **Role of $\tanh$:** The derivative of $\tanh$ satisfies $0 < \tanh'(z) = 1 - \tanh^2(z) \leq 1$. In practice, activations frequently saturate into regimes where $\tanh'(z) \ll 1$ (e.g., 0.1 or 0.2). Even if $\mathbf{W}_{hh} \approx 1$, multiplying factors of ≤ 0.5 across 10 steps drives the gradient to $(0.5)^{10} \approx 0.00097 \to \mathbf{0}$.

3. **Gradient Norm Clipping:** When $\|\mathbf{g}\|_2 > \theta$:

$$\mathbf{g}_{\text{clipped}} = \theta \frac{\mathbf{g}}{\|\mathbf{g}\|_2} = 5.0 \frac{\mathbf{g}}{25.0} = \mathbf{0.2} \cdot \mathbf{g}$$

The gradient vector's direction is preserved perfectly, while its magnitude is scaled down from 25.0 to exactly 5.0.

Solution 15: LSTM Cell Gates & CEC Mathematical Proof

1. **Core Equations:**

   - $\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$

- $\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
- $\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$
- $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$
- $\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$
- $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$

2. **Partial Derivative:** Differentiating $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$ with respect to $\mathbf{C}_{t-1}$:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \text{diag}(\mathbf{f}_t) + \mathbf{C}_{t-1} \frac{\partial \mathbf{f}_t}{\partial \mathbf{C}_{t-1}} + \frac{\partial (\mathbf{i}_t \odot \tilde{\mathbf{C}}_t)}{\partial \mathbf{C}_{t-1}}$$

Directly along the linear highway:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} \approx \mathbf{f}_t$$

3. **Constant Error Carousel (CEC) Proof:** Across T time steps, the gradient of the cell state propagates as:

$$\frac{\partial \mathbf{C}_T}{\partial \mathbf{C}_1} = \prod_{t=2}^{T} \frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} \approx \prod_{t=2}^{T} \mathbf{f}_t$$

If the network learns to keep the forget gate open ($\mathbf{f}_t \approx 1$ for critical memory channels), the product is:

$$\prod_{t=2}^{T} 1.0 = 1.0$$

The gradient flows through the cell state highway **without exponential decay or explosion**! This is the Constant Error Carousel that allows LSTMs to bridge dependencies spanning thousands of time steps.